Forge — Backend Integration Reference

This documents what's **already built** as a mock layer so a backend developer can see the exact shape of every call without reading the UI code. Nothing here is new — it's a summary of `src/services/forge.ts` (the mock "backend") and the `DEV NOTE` comments already sitting above each action handler in `src/routes/forge.tsx`.

How it's wired today

Every Forge action (Fusion, Level Up, Reroll, Identity Swap, Dismantle) already goes through **one function**, not ad-hoc local state changes:

```
executeForgeActionPayload(actionType, itemId, options) →
Promise<ActionResult>
```

This lives in `src/services/forge.ts`. The UI never mutates an item's stats, rarity, level, or name directly — it calls this function, awaits a result object, and only then updates what's on screen. That's the seam a backend developer replaces: swap the *inside* of `executeForgeActionPayload` (and the 5 functions it delegates to) for real `fetch()` calls. The call sites in `forge.tsx` don't need to change.

Each handler in `forge.tsx` already has the exact intended REST shape commented directly above the mock call, e.g.:

```
// DEV NOTE: Replace mock logic with backend API call
// Endpoint: POST /api/forge/reroll
// Payload: { userId, itemId: activeItem.id, xpCost: 5000 }
console.log("[API PREVIEW] POST /api/forge/reroll Payload:", payload);
```

The 5 actions

Action	Endpoint (proposed)	Mock function	What can change i response
Fusion	`POST /api/forge/fuse`	`fuseItems()`	**rarity**, raidPower, forgeable/rerollabl
Level Up	`POST /api/forge/upgrade`	`upgradeItem()`	**level**, XP cost scale current level

Action	Endpoint (proposed)	Mock function	What can change i response
Stat Reroll	`POST /api/forge/reroll`	`rerollItemStatValues()`	**stats** (activity/consisten via an 80–100% qu
Identity Swap	`POST /api/forge/identity-swap`	`swapItemIdentity()`	**item name**, image/ stats — swaps to a item template in th slot + rarity
Dismantle	`POST /api/forge/dismantle`	`dismantleDuplicateItem()`	removes the item, r XP

Equip/unequip (which also changes what's shown — name, rarity badge, frame overlay) goes through the separate gear-switch flow, not through `executeForgeActionPayload`.

Request / response shape (already typed)

These interfaces are already defined in `src/services/forge.ts` — this is not a proposal, it's what the mock function actually returns today:

```
export interface ForgeRerollResult {
  success: boolean;
  item?: Item;          // full updated item, including new .stats
  costXP?: number;
  qualityRoll?: number; // 0.8–1.0, drives the "% MAX POTENTIAL" badge
  updatedPlayer?: Player;    // new spendableXP balance
  updatedInventory?: Item[]; // full inventory after the change
  error?: string;
}
```

Fusion, Upgrade, Dismantle, and Identity Swap each have their own `Forge*Result` interface right next to this one, following the same pattern: `success`, the changed `item` / `newItem`, `updatedPlayer`, `updatedInventory`, optional `error`.

Where the "roll a range, then reveal it" behavior already lives

This is the part of your message about reroll "pulling a number between a range in the background, then showing the new numbers" — it's already implemented exactly that way, not just displayed as if it's happening:

1. `handleStatRerollAction` (forge.tsx) fires the slot-machine spin animation immediately (`isRollingStats = true` — this is what makes the stat cards show scrambling random numbers).
2. After a fixed 1.2s animation window, it calls `executeForgeActionPayload("reroll", itemId)` .
3. Inside the mock, `rerollItemStatValues()` (services/forge.ts) picks a real `qualityRoll` between `FORGE_REROLL_RULES.statReroll.minQuality` (0.8) and `.maxQuality` (1.0), and computes the item's new `stats` from it.
4. The UI takes the *returned* `item.stats` — not a locally-guessed number — and that's what the stat cards settle on when the spin stops.

A backend developer replacing step 3 with a real endpoint just needs to return the same `ForgeRerollResult` shape; steps 1, 2, and 4 don't change.

Rule tables the backend should treat as the source of truth

`src/config/forgeConfig.ts` already has the exact numbers (duplicate counts, XP costs, dismantle refunds, quality-roll ranges) as plain exported constants (`FORGE_UPGRADE_RULES` , `FORGE_DISMANTLE_RATES` , `FORGE_REROLL_RULES`). These are the numbers currently driving both the mock and the UI's own client-side validation (e.g. "need 2 more duplicates"). Whoever builds the real endpoints should treat this file as the spec for those values, then keep it in the client only as the copy used for optimistic UI/validation — server response is still authoritative.

What's left to change (and what isn't)

Change: the inside of the 5 functions in `src/services/forge.ts` — swap the local array mutation + `setMockInventory` / `setMockPlayer` calls for a real `fetch(...)` to your endpoint, returning the same result shape.

Don't need to change: `forge.tsx` — every action already calls through `executeForgeActionPayload` , awaits it, and drives all animation/UI state off the returned `item` / `updatedPlayer` / `updatedInventory` . As long as the real endpoint returns the same shape, the UI layer doesn't need touching.